

EXPERIMENT 08 – DATA-DRIVEN AUTOMATION (DDT)

1. Module Name and Experiment Title

Module Name: Module 2 – Unit Test Frameworks

Experiment Number: 08

Experiment Title: Data-Driven Automation (DDT)

Programming Language: Python

Automation Tool: Selenium WebDriver

Website: SauceDemo

Website URL: <https://www.saucedemo.com/>

2. Problem Statement

Testing a login page with multiple sets of credentials manually can be repetitive and time-consuming. To make the process more efficient, data-driven automation can be used to execute multiple test cases using a single automation script.

In this experiment, Selenium WebDriver and Python are used to automate login testing on the SauceDemo website. Test data is stored in an external CSV file, which the script reads to execute different login scenarios. The results are verified using assertions, and screenshots are captured to document the execution.

3. Objective

1. Implement data-driven testing using Selenium WebDriver and Python.
 2. Store multiple login test cases in an external CSV file.
 3. Read test data from the CSV file and execute login scenarios automatically.
 4. Verify successful and unsuccessful login attempts.
 5. Use assertions to validate the expected results.
 6. Capture screenshots of login details and test results.
 7. Display the execution status of each test case in the terminal.
-

4. Tools, Software and Concepts Used

Tools and Software

- Python 3.10
- Visual Studio Code

- Selenium WebDriver
- Google Chrome
- CSV file

Concepts Used

- Data-Driven Testing (DDT)
 - CSV file handling
 - Selenium WebDriver
 - Explicit waits
 - Element locators
 - Assertions
 - Screenshot capturing
 - Exception handling
 - Automated browser control
-

5. Source Code / Implementation Steps

A. Test Data (`test_data.csv`)

The CSV file contains three login test cases. Each test case includes a username, password, and expected result.

```
username,password,expected_result
standard_user,secret_sauce,success
invalid_user,wrong_password,error
locked_out_user,secret_sauce,error
```

Explanation:

- `standard_user`: Valid credentials for a successful login.
- `invalid_user`: Invalid credentials to test the login error message.
- `locked_out_user`: Credentials for testing the locked-out account scenario.
- `expected_result`: Defines whether the login is expected to succeed or fail.

B. Python Source Code (`exp_08_data_driven_automation.py`)

```
import csv
import os
import time
from pathlib import Path

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

os.makedirs("screenshots", exist_ok=True)

CSV_FILE = Path(__file__).resolve().parent / "test_data.csv"
```

```

BASE_URL = "https://www.saucedemo.com/"

with open(CSV_FILE, mode="r", newline="", encoding="utf-8") as file:
    test_cases = list(csv.DictReader(file))

for index, test_case in enumerate(test_cases, start=1):
    username = test_case["username"]
    password = test_case["password"]
    expected_result = test_case["expected_result"]

    driver = webdriver.Chrome()
    wait = WebDriverWait(driver, 15)

    try:
        driver.maximize_window()
        driver.get(BASE_URL)

        print(f"\nTest Case {index}: {username}")
        print("SauceDemo website opened successfully.")
        time.sleep(3)

        username_field = wait.until(
            EC.visibility_of_element_located((By.ID, "user-name"))
        )
        username_field.send_keys(username)

        password_field = driver.find_element(By.ID, "password")
        password_field.send_keys(password)
        time.sleep(3)

        driver.save_screenshot(
            f"screenshots/08_test_case_{index}_login.png"
        )
        print(f"Screenshot {index}: Login details saved.")

        login_button = driver.find_element(By.ID, "login-button")
        login_button.click()
        time.sleep(3)

        if expected_result == "success":
            wait.until(
                EC.visibility_of_element_located(
                    (By.CLASS_NAME, "inventory_list")
                )
            )

            assert "/inventory.html" in driver.current_url
            print("Actual result: Login successful.")
            print("Test status: PASS")

        else:
            error_message = wait.until(
                EC.visibility_of_element_located(
                    (By.CSS_SELECTOR, "h3[data-test='error']")
                )
            ).text

            assert error_message != ""
            print("Actual result: Login failed as expected.")
            print("Error message:", error_message)
            print("Test status: PASS")
    
```

```

        driver.save_screenshot(
            f"screenshots/08_test_case_{index}_result.png"
        )
        print(f"Screenshot {index}: Test result saved.")

    except Exception as error:
        print("Test status: FAIL")
        print("Error:", error)

        driver.save_screenshot(
            f"screenshots/08_test_case_{index}_failure.png"
        )

    finally:
        driver.quit()
        print("Browser closed successfully.")
        time.sleep(2)

print("\nData-driven automation completed.")

```

C. Implementation Steps

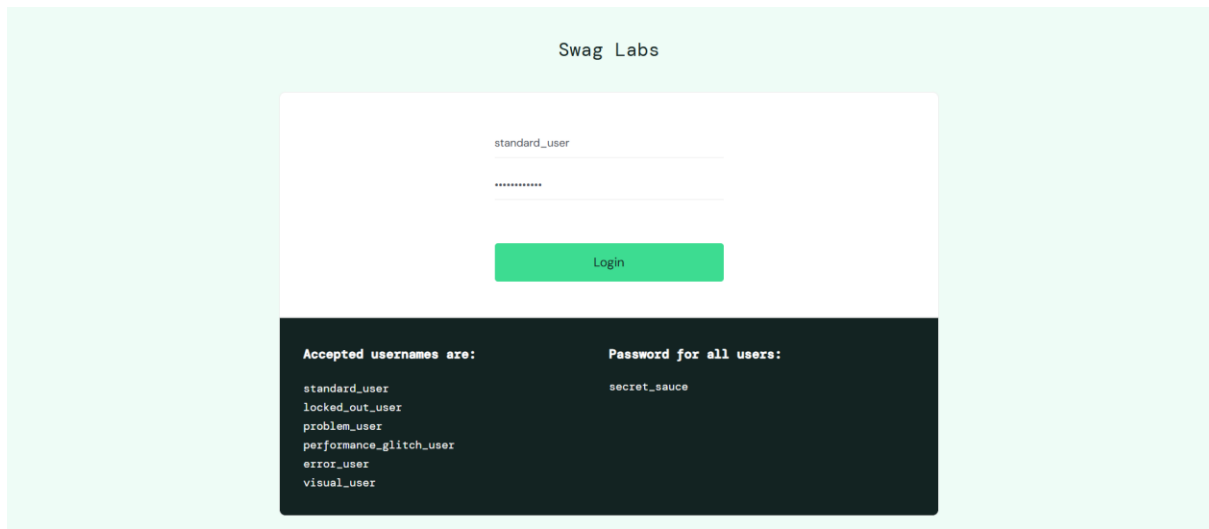
1. Create a CSV file named `test_data.csv` and enter the login test data.
2. Create the Python script `exp_08_data_driven_automation.py`.
3. Import the required Selenium and Python libraries.
4. Read the test cases from the CSV file using `csv.DictReader`.
5. Launch Chrome and open the SauceDemo login page for each test case.
6. Enter the username and password from the CSV file.
7. Click the Login button to submit the credentials.
8. Verify the result according to the expected outcome.
9. Capture screenshots of the login details and test results.
10. Display the test status in the terminal.
11. Close the browser after each test case.

6. Output with Clearly Labelled Screenshots

The automation script was executed successfully with three test cases. The screenshots below document the login process and the results of each test case.

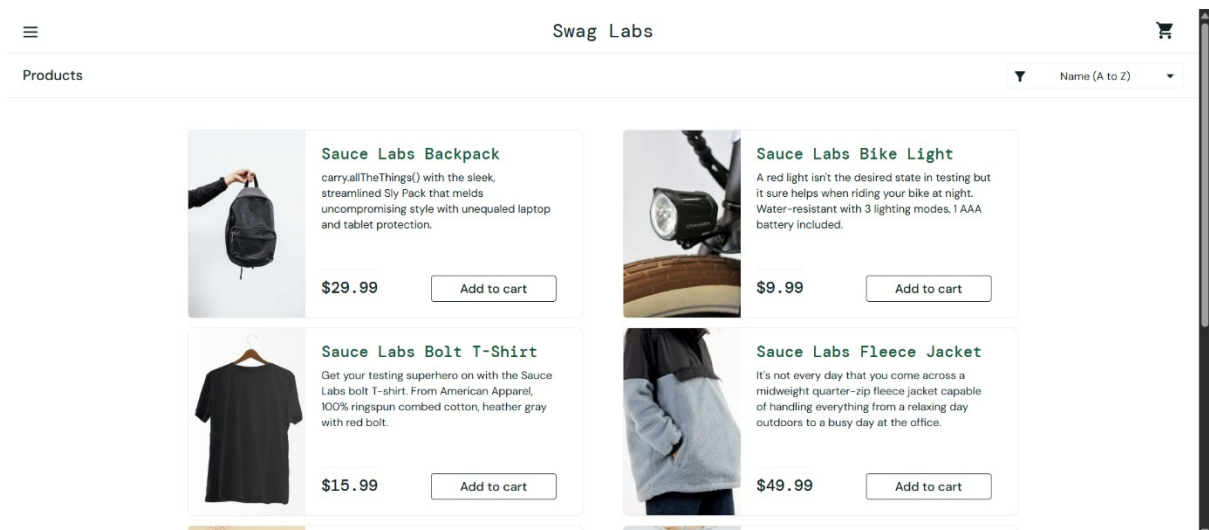
Screenshot 1: Valid User Login Details

Description: The SauceDemo login page displays the username `standard_user` and the corresponding password before the login button is clicked.



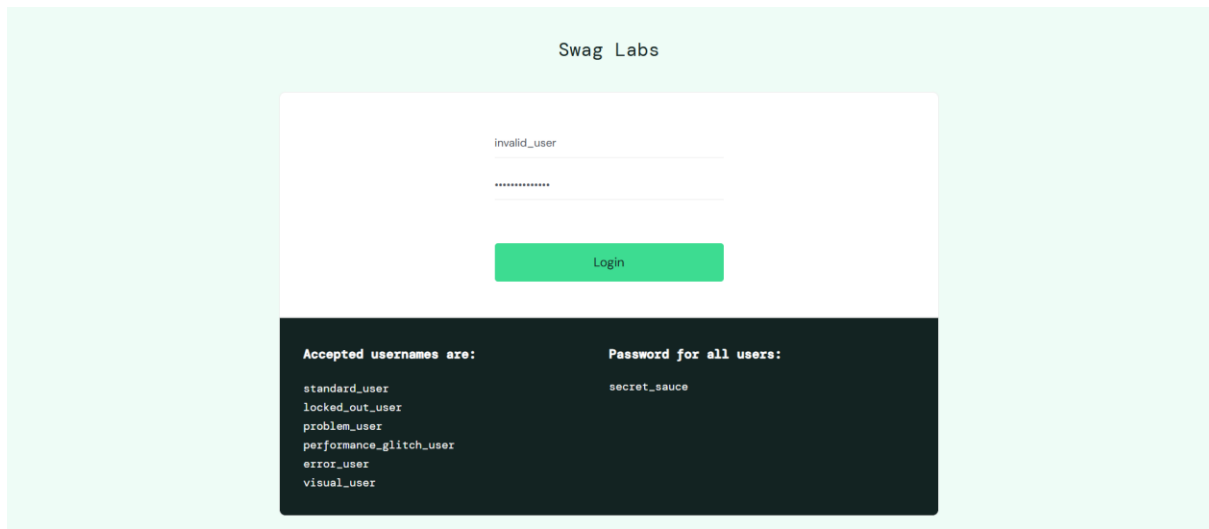
Screenshot 2: Successful Login and Products Page

Description: The valid credentials were accepted, and the browser navigated to the Products page. This confirms that the successful login scenario passed.



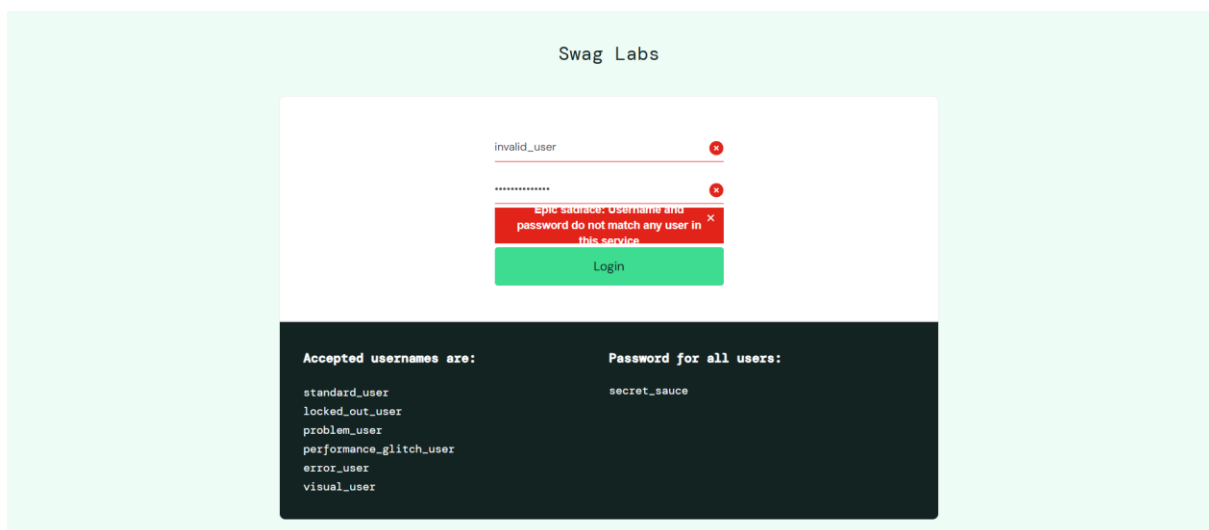
Screenshot 3: Invalid User Login Details

Description: The SauceDemo login page displays the username `invalid_user` and its corresponding password before submitting the login form.



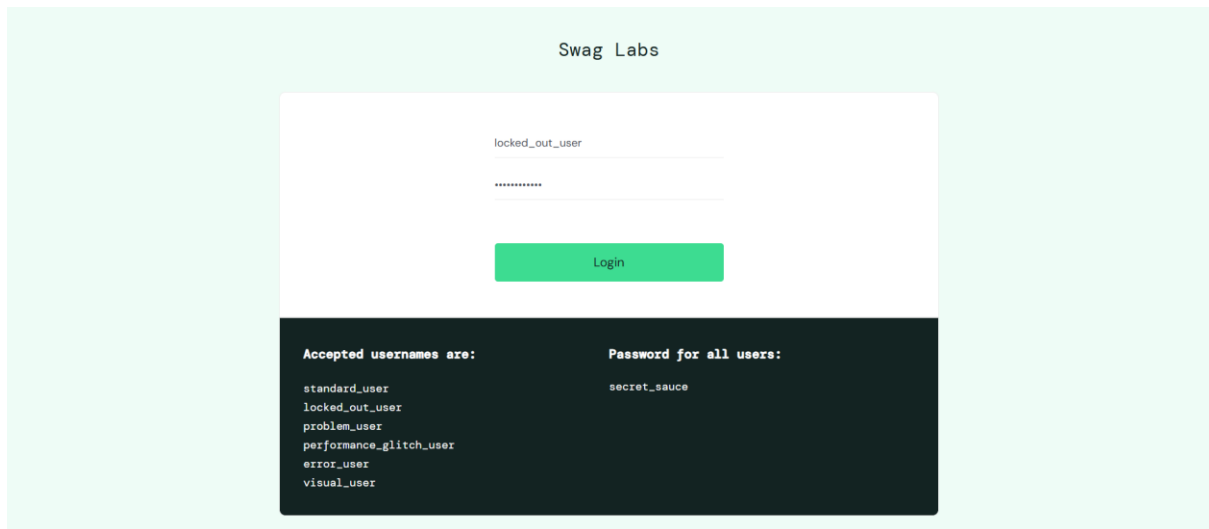
Screenshot 4: Invalid Login Error Message

Description: The application displayed an error message because the provided username and password did not match any user. The automation verified that the login failed as expected.



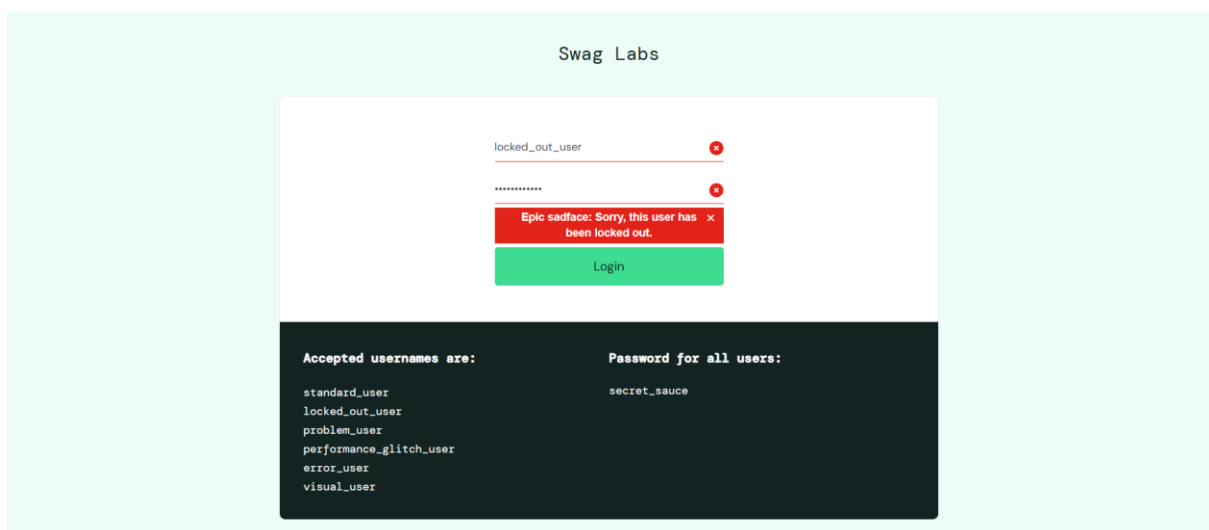
Screenshot 5: Locked-Out User Login Details

Description: The login page displays the username `locked_out_user` and its corresponding password before the login attempt.



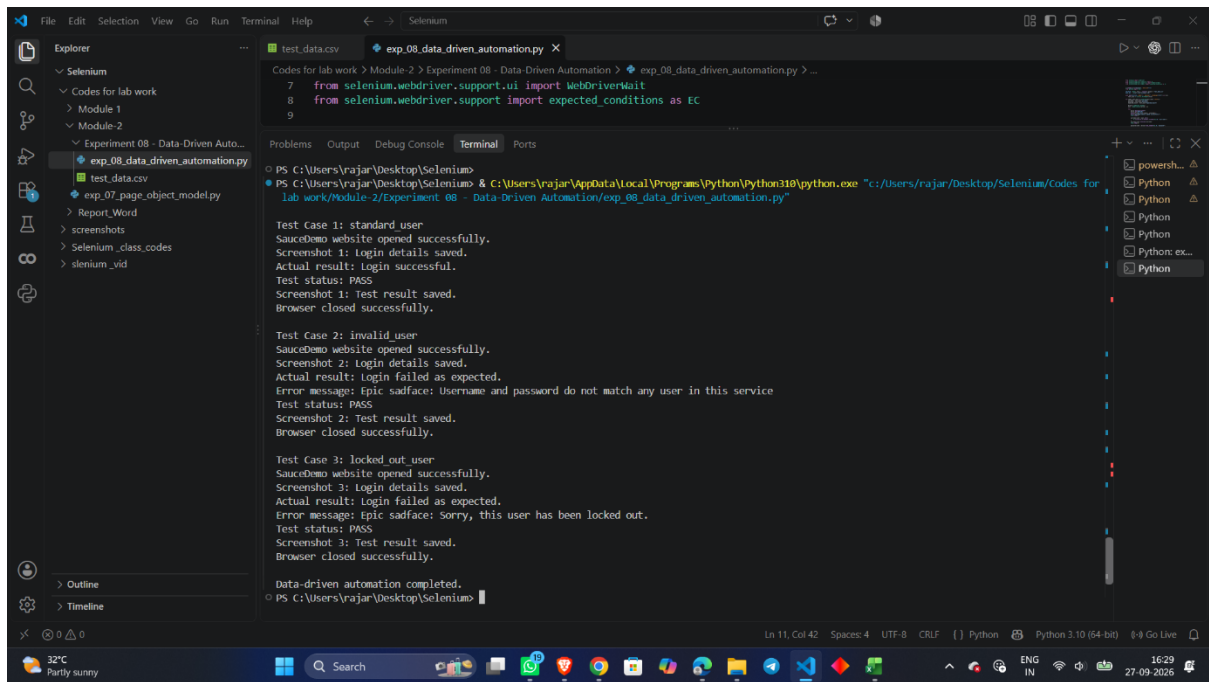
Screenshot 6: Locked-Out User Error Message

Description: The application displayed an error message indicating that the user account had been locked out. The automation verified that the login failed as expected.



Screenshot 7: Console Output – Data-Driven Test Execution

Description: The VS Code terminal displays the execution results of all three test cases. The `standard_user`, `invalid_user`, and `locked_out_user` test cases all received a PASS status. The output also confirms that the browser closed successfully after each test case and that data-driven automation was completed.



Console Output

Test Case 1: standard_user
SauceDemo website opened successfully.
Screenshot 1: Login details saved.
Actual result: Login successful.
Test status: PASS
Screenshot 1: Test result saved.
Browser closed successfully.

Test Case 2: invalid_user
SauceDemo website opened successfully.
Screenshot 2: Login details saved.
Actual result: Login failed as expected.
Error message: Epic sadface: Username and password do not match any user in this service
Test status: PASS
Screenshot 2: Test result saved.
Browser closed successfully.

Test Case 3: locked_out_user
SauceDemo website opened successfully.
Screenshot 3: Login details saved.
Actual result: Login failed as expected.
Error message: Epic sadface: Sorry, this user has been locked out.
Test status: PASS
Screenshot 3: Test result saved.
Browser closed successfully.

Data-driven automation completed.

7. Result, Observation and Conclusion

Result

The data-driven automation script was executed successfully using Selenium WebDriver and Python. All three login test cases were completed, and each received a PASS status.

Observation

1. The `standard_user` account successfully logged in and reached the Products page.
2. The `invalid_user` account produced the expected invalid-credentials error message.
3. The `locked_out_user` account produced the expected account lockout error message.
4. The automation script read test data from the external CSV file and executed each test case automatically.
5. Screenshots were captured to document the login details and test results.
6. The browser closed automatically after each test case.
7. The terminal displayed the execution status of all three test cases.

Conclusion

The experiment successfully demonstrated Data-Driven Automation using Selenium WebDriver and Python. By storing test data in an external CSV file, multiple login scenarios were executed using a single automation script.

The experiment also demonstrated how assertions, explicit waits, and screenshot capturing can be used to verify and document automated test results. All three test cases passed according to their expected outcomes.